EXP 801 DS Wheat Heat Drought

— Ed Chaplin (USyd), *Apr–June 2025* —

Li-6800 STOMATAL CONDUCTANCE MODEL PARAMETER ESTIMATION FOR A - LIGHT OFF TO LIGHT ON (INCREASE)

- Using the final model fitted to the data, obtain the parameters of interest
- These include:
- A0, being the starting value of the A response (intercept/lower asymptote)
- A1, being the upper asymptote of the A responsE
-

Load the libraries

```
library(knitr)
#knitr::spin("Exp801.li6800.stomatalCondParamEstimation_A_i.v1.r")
library(ggplot2)
opts_chunk$set("tidy" = FALSE, "dev" = "png", "dpi" = 300,
               "fig.width" = 7.5, "fig.height" = 10, "comment"=NA, fig.path = "figures/increase/")
library(dae)
library(dplyr)
library(brms) ##package used to fit bayesian regression model using Stan
library(tidyr)
library(emmeans)
library(tidybayes)
library(pracma)
```

Set up constants and files

```
options(width=90)
load("../data/stomatalCondModel_A_i.v1.Rdata")
saveFile <- "../data/stomatalCondParamEstimation_A_i.v1.Rdata"
```

Have a look at the final model summary and store fixed effects

```
summary(fit_on_A_final)
```

```
 Family: gaussian
  Links: mu = identity
Formula: A ~ g0 + (g1 - g0) * exp(-exp(((-tmin)/k) + 1))
         g0 ~ 1 + Room_Temperature * Watering + (1 | Rep)
         g1 ~ 1 + Room_Temperature * Watering + (1 | Rep)
         k ~ 1 + Room_Temperature + Watering
   Data: li6800.on.dat (Number of observations: 1462)
  Draws: 6 chains, each with iter = 10000; warmup = 6000; thin = 1;
         total post-warmup draws = 24000

Multilevel Hyperparameters:
~Rep (Number of levels: 3)
                Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(g0_Intercept)     1.39      0.25     0.95     1.93 1.00    13804    16268
sd(g1_Intercept)     1.00      0.41     0.44     2.01 1.00    11896    13652
```

```
Regression Coefficients:
                                         Estimate Est.Error l-95% CI u-95% CI Rhat
g0_Intercept                                 0.46      0.10     0.27     0.66 1.00
g0_Room_TemperatureCER8_Amb                  0.46      0.10     0.27     0.65 1.00
g0_WateringUnlim                             0.67      0.10     0.48     0.86 1.00
g0_Room_TemperatureCER8_Amb:WateringUnlim    0.66      0.10     0.46     0.85 1.00
g1_Intercept                                24.15      0.53    23.18    25.28 1.00
g1_Room_TemperatureCER8_Amb                 -2.37      0.22    -2.81    -1.94 1.00
g1_WateringUnlim                             1.48      0.22     1.07     1.90 1.00
g1_Room_TemperatureCER8_Amb:WateringUnlim    3.15      0.29     2.59     3.71 1.00
k_Intercept                                  3.71      0.10     3.53     3.90 1.00
k_Room_TemperatureCER8_Amb                   3.34      0.16     3.02     3.65 1.00
k_WateringUnlim                              1.16      0.14     0.89     1.45 1.00
                                         Bulk_ESS Tail_ESS
g0_Intercept                                30110    16523
g0_Room_TemperatureCER8_Amb                 31234    17946
g0_WateringUnlim                            30890    17084
g0_Room_TemperatureCER8_Amb:WateringUnlim   30943    18423
g1_Intercept                                 9940    12894
g1_Room_TemperatureCER8_Amb                 17721    17928
g1_WateringUnlim                            18463    18378
g1_Room_TemperatureCER8_Amb:WateringUnlim   18010    17784
k_Intercept                                 22420    18011
k_Room_TemperatureCER8_Amb                  22088    19103
k_WateringUnlim                             21697    16391

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     2.52      0.05     2.43     2.62 1.00    27090    17465

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

Build a dataframe of model parameters

```
li6800.A.i.param <- expand.grid(Room_Temperature = levels(li6800.on.dat$Room_Temperature),
                                Watering = levels(li6800.on.dat$Watering))
```

Use the emmeans package to get out the expected parameter estimates, along with HPD (credible intervals)

```
g0.A.emmeans <- as.data.frame(emmeans(fit_on_A_final,
                                      ~ Room_Temperature * Watering,
                                      nlpar = "g0"))
g0.A.emmeans <- g0.A.emmeans %>% rename_with(~paste0("g0_A_", .x),
                                             -all_of(c("Room_Temperature",
                                                       "Watering")))
g0.A.emmeans <- within(g0.A.emmeans, {
  g0_A_tabquant <- paste0(round(g0_A_emmean, 3), " [",
                          round(g0_A_lower.HPD, 3), ", ",
                          round(g0_A_upper.HPD, 3), "]")
```

```
})
li6800.A.i.param <- left_join(li6800.A.i.param, g0.A.emmeans)
```

```
Joining with `by = join_by(Room_Temperature, Watering)`
```

```
g1.A.emmeans <- as.data.frame(emmeans(fit_on_A_final,
                                      ~ Room_Temperature * Watering,
                                      nlpar = "g1"))
g1.A.emmeans <- g1.A.emmeans %>% rename_with(~paste0("g1_A_", .x),
                                             -all_of(c("Room_Temperature",
                                                       "Watering")))
g1.A.emmeans <- within(g1.A.emmeans, {
  g1_A_tabquant <- paste0(round(g1_A_emmean, 3), " [",
                          round(g1_A_lower.HPD, 3), ", ",
                          round(g1_A_upper.HPD, 3), "]")

})
li6800.A.i.param <- left_join(li6800.A.i.param, g1.A.emmeans)
```

```
Joining with `by = join_by(Room_Temperature, Watering)`
```

```
k.A.emmeans <- as.data.frame(emmeans(fit_on_A_final,
                                      ~ Room_Temperature * Watering,
                                      nlpar = "k"))
k.A.emmeans <- k.A.emmeans %>% rename_with(~paste0("k_A_", .x),
                                           -all_of(c("Room_Temperature",
                                                     "Watering")))
k.A.emmeans <- within(k.A.emmeans, {
  k_A_tabquant <- paste0(round(k_A_emmean, 3), " [",
                         round(k_A_lower.HPD, 3), ", ",
                         round(k_A_upper.HPD, 3), "]")

})
li6800.A.i.param <- left_join(li6800.A.i.param, k.A.emmeans)
```

```
Joining with `by = join_by(Room_Temperature, Watering)`
```

Calculate the time taken to reach 95% Amax

t = -k(log(-log(0.95*g1-g0/g1-g0))-1)

```
g0_A_draws <- emmeans(fit_on_A_final,
                      ~ Room_Temperature * Watering,
                      nlpar = "g0") %>%
  gather_emmeans_draws()

g1_A_draws <- emmeans(fit_on_A_final,
                      ~ Room_Temperature * Watering,
                      nlpar = "g1") %>%
  gather_emmeans_draws()

k_A_draws <- emmeans(fit_on_A_final,
                     ~ Room_Temperature * Watering,
                     nlpar = "k") %>%
  gather_emmeans_draws()
```

```
combined_draws <- g0_A_draws %>%
  rename(g0 = .value) %>%
  left_join(g1_A_draws %>% rename(g1 = .value),
            by = c(".draw", "Room_Temperature", "Watering")) %>%
  left_join(k_A_draws %>% rename(k = .value),
            by = c(".draw", "Room_Temperature", "Watering"))

combined_draws <- combined_draws %>%
  mutate(t95 = -k*(log(-1*log((0.95*g1 - g0)/(g1-g0)))-1))

summary_t95 <- combined_draws %>%
  group_by(Room_Temperature, Watering) %>%
  median_hdi(t95) %>%
  rename(t95_emmean = t95) %>%
  rename(t95_lower.HPD = .lower) %>%
  rename(t95_upper.HPD = .upper)

summary_t95 <- within(summary_t95, {
  t95_tabquant <- paste0(round(t95_emmean, 3), " [",
                         round(t95_lower.HPD, 3), ", ",
                         round(t95_upper.HPD, 3), "]")

})
li6800.A.i.param <- left_join(li6800.A.i.param, summary_t95)
```

```
Joining with `by = join_by(Room_Temperature, Watering)`
```

Using t95 calculated above, use this to determine A95

Use the individual t95 values calculated above in combined_draws A ~ g0 + (g1 - g0) * exp(-exp(((-tmin)/k)+1)),

```
combined_draws <- combined_draws %>%
  mutate(A95 = g0 + (g1 - g0) * exp(-exp(((-t95)/k)+1)),)

summary_A95 <- combined_draws %>%
  group_by(Room_Temperature, Watering) %>%
  median_hdi(A95) %>%
  rename(A95_emmean = A95) %>%
  rename(A95_lower.HPD = .lower) %>%
  rename(A95_upper.HPD = .upper)

summary_A95 <- within(summary_A95, {
  A95_tabquant <- paste0(round(A95_emmean, 3), " [",
                         round(A95_lower.HPD, 3), ", ",
                         round(A95_upper.HPD, 3), "]")

})
li6800.A.i.param <- left_join(li6800.A.i.param, summary_A95)
```

```
Joining with `by = join_by(Room_Temperature, Watering, .width, .point, .interval)`
```

Next, calculate the %Limitation. This involves integrating the nonlinear function

Can calculate this using the contents of combined_draws as well!

```r
##define the antiderivative of our function
Aint <- function(tmin, g0, g1, k){
  g0*tmin - k*(g1-g0)*expint::expint_Ei(-exp(1 - tmin/k))
}

##get out the total integral, being the function evaluated between t=0 and t= max(tmin)
combined_draws <- combined_draws %>%
  mutate(Aint.tot.0 = Aint(0, g0, g1, k),
         Aint.tot.max = Aint(max(li6800.on.dat$tmin), g0, g1, k)) %>%
  mutate(Aint.tot.diff = Aint.tot.max - Aint.tot.0)
head(combined_draws %>% select(Aint.tot.0, Aint.tot.max, Aint.tot.diff))
```

```
Adding missing grouping variables: `Room_Temperature`, `Watering`
```

```
# A tibble: 6 × 5
# Groups:   Room_Temperature, Watering [1]
  Room_Temperature Watering Aint.tot.0 Aint.tot.max Aint.tot.diff
  <fct>            <fct>         <dbl>        <dbl>         <dbl>
1 CER7_Heat        Lim            1.61        1244.         1242.
2 CER7_Heat        Lim            1.60        1251.         1250.
3 CER7_Heat        Lim            1.61        1262.         1260.
4 CER7_Heat        Lim            1.73        1341.         1339.
5 CER7_Heat        Lim            1.61        1321.         1319.
6 CER7_Heat        Lim            1.68        1319.         1318.
```

```r
##next, evaluate the integral of the difference between Amax and A, for the interval
##between 0 and t95
Aint.Amax <- function(tmin, g0, g1, k){
  g1*tmin - (g0*tmin - k*(g1-g0)*expint::expint_Ei(-exp(1 - tmin/k)))
}

##now evaluate the integral using Aint.Amax
combined_draws <- combined_draws %>%
  mutate(Aint.Amax.0 = Aint.Amax(0, g0, g1, k),
         Aint.Amax.t95 = Aint.Amax(t95, g0, g1, k)) %>%
  mutate(Aint.Amax.diff = Aint.Amax.t95 - Aint.Amax.0)

##calculate the ratio of Aint.Amax.diff and Aint.tot.diff
combined_draws <- combined_draws %>%
  mutate(Limitation = Aint.Amax.diff/Aint.tot.diff)

summary_limitation <- combined_draws %>%
  group_by(Room_Temperature, Watering) %>%
  median_hdi(Limitation) %>%
  rename(Limit_emmean = Limitation) %>%
  rename(Limit_lower.HPD = .lower) %>%
  rename(Limit_upper.HPD = .upper)

summary_limitation <- within(summary_limitation, {
  Limit_tabquant <- paste0(round(Limit_emmean*100, 3), " [",
                           round(Limit_lower.HPD*100, 3), ", ",
                           round(Limit_upper.HPD*100, 3), "]")

})
li6800.A.i.param <- left_join(li6800.A.i.param, summary_limitation)
```

```
Joining with `by = join_by(Room_Temperature, Watering, .width, .point, .interval)`
```

Format this in a nicer way for export to Excel

```
li6800.A.i.export <- li6800.A.i.param %>% select(Room_Temperature, Watering,
                                                 grep("tabquant", colnames(.))) %>%
  pivot_longer(cols = 3:ncol(.),
               names_to = "params",
               values_to = "values") %>%
  pivot_wider(names_from = c("Room_Temperature", "Watering"),
              values_from = "values")
li6800.A.i.export
```

```
# A tibble: 6 × 5
  params         CER7_Heat_Lim           CER8_Amb_Lim       CER7_Heat_Unlim CER8_Amb_Unlim
  <chr>          <chr>                   <chr>              <chr>           <chr>
1 g0_A_tabquant  0.461 [0.273, 0.66]     0.924 [0.637, 1.1... 1.132 [0.854, ... 2.252 [1.869,...
2 g1_A_tabquant  24.121 [23.189, 25.288] 21.749 [20.776, 2... 25.602 [24.624... 26.387 [25.37...
3 k_A_tabquant   3.707 [3.526, 3.896]    7.043 [6.741, 7.3... 4.872 [4.622, ... 8.206 [7.835,...
4 t95_tabquant   14.645 [13.917, 15.384] 27.647 [26.451, 2... 19.115 [18.105... 31.827 [30.36...
5 A95_tabquant   22.915 [21.988, 23.988] 20.662 [19.737, 2... 24.322 [23.404... 25.067 [24.10...
6 Limit_tabquant 10.456 [9.878, 11.061]  21.339 [20.204, 2... 13.793 [12.962... 24.316 [22.82...
```

Save data and export

```
save.image(saveFile)
write.csv(li6800.A.i.export, "../data/li6800.A.i.param.export.csv", row.names = FALSE)
```